

TruthLens — Presentation Transcript

Duration: ~10 Minutes | **Members:** 4 | **Subject:** Data Mining

MEMBER 1 — Introduction & Problem Statement

 (0:00 – 1:30)

Good morning everyone. My name is [Member 1], and today our team is presenting **TruthLens** — a fake news detection system powered entirely by data mining, NLP, and real-time web scraping.

We live in the age of information overload. Every day, millions of news articles, social media posts, and blog entries flood the internet — and not all of them are true. According to a 2023 Reuters Institute report, over **60% of people** have encountered fake news in the past week alone. Misinformation spreads six times faster than real news on social media, with consequences ranging from election interference to public health panic.

The question we asked ourselves is — **can a machine learn to detect fake news the way a trained journalist would?** And furthermore — can we continuously mine fresh, real-world news data to cross-verify claims in real time?

The answer is **yes**. TruthLens is our answer. It does two things: it classifies whether a news article is real or fake using a trained Naive Bayes model — and it cross-verifies that text against a live database of scraped, real-world headlines, populated automatically from eight trusted news sources using our own multi-source web scraper.

I'll now hand it over to [Member 2], who will cover the full data collection and mining pipeline.

MEMBER 2 — Dataset, Data Scraping & Data Mining Core

 (1:30 – 5:00)

Thank you [Member 1]. I'm [Member 2], and I'll be covering the foundation of everything — **where the data comes from, how we scraped it, and how we mine it**.

Part A — The Training Dataset

For training our classifier, we used the **Kaggle Fake and Real News Dataset** — one of the most widely referenced open-source datasets in the NLP community for misinformation research.

The dataset contains approximately **44,000 labeled news articles**, evenly split between real and fake. Each record has three key fields:

- **title** — the headline of the article
- **text** — the full body content
- **label** — 0 for REAL, 1 for FAKE

The fake articles were sourced from outlets flagged by PolitiFact and other fact-checking bodies. The real articles came from reputable wire services like Reuters. This balance is critical — a skewed dataset would produce a biased classifier.

A key data engineering decision: we **combined title and text into a single string** for the article model. Deceptive language in fake news is often packed into the headline itself — merging them preserves that signal. For the headline model, we trained separately on titles only — with a different TF-IDF configuration, since headlines are short and removing stop words hurts more than it helps.

Part B — Live News Data Scraping (bulk_fetch.py)

But we didn't stop at a static dataset. We built a **multi-source, automated web scraper** — `bulk_fetch.py` — that continuously populates a live SQLite database with verified, real-world headlines. This is a core data mining component of TruthLens.

How it works:

Source 1 — Inshorts (HTML Scraping): Our `news_fetcher.py` module uses `requests` and `BeautifulSoup` to directly scrape the Inshorts website across 7 categories — national, business, politics, technology, world, science, and sports. We target the stable `itemprop="headline"` attribute, making the scraper resilient to CSS class changes. This yields approximately 70 headlines per run.

Source 2 — RSS Feeds (XML Parsing): Our bulk fetcher consumes 33 live RSS feeds from 8 trusted sources — BBC News, Al Jazeera, CNN, NPR, The Hindu, NDTV, Times of India, India Today, TechCrunch, and Google News. We parse the `<item>` tags using Python's `xml.etree.ElementTree`, with a BeautifulSoup fallback for malformed feeds.

The scraper is smart — it:

- **Deduplicates** headlines on insert using SQLite's `SELECT 1 WHERE title = ?` check
- **Stops early** once the target headline count is reached
- **Sleeps 300ms between requests** to comply with polite crawling norms
- Uses **chunked text splitting** at sentence boundaries for long translations

In our last run, we went from **70 to 443 headlines** — exceeding our 400-headline target — in under 30 seconds, from 8 live news sources simultaneously.

The SQLite schema is simple and efficient:

```
headlines (id, title, category, source, url, fetched_at)
```

An index on `title` makes deduplication $O(\log n)$ instead of a full table scan.

Part C — The Five Data Mining Phases

This entire pipeline maps directly to the **five phases of data mining**:

Phase 1 — Data Collection. We collect from two channels: the static Kaggle dataset for training, and our live scraper for real-time cross-verification. Together they represent structured and semi-structured

data ingestion.

Phase 2 — Data Cleaning. Raw scraped text contains URLs, HTML tags, emails, special characters, and extra whitespace. Our `TextPreprocessor` class strips all of this using regular expressions before any further processing.

Phase 3 — Feature Engineering with TF-IDF. This is where data mining becomes powerful. We transform raw text into **TF-IDF feature vectors** — a sparse numerical representation where each dimension corresponds to a term in the vocabulary. TF-IDF weights a term higher if it appears frequently in one document but rarely across the corpus — so it captures discriminative signals, not common filler words. We use `ngram_range=(1, 2)` to capture not just individual words but word pairs like "breaking news" or "secretly filmed". With `max_features=5000`, we mine the 5000 most informative terms.

Phase 4 — Pattern Discovery via Naive Bayes. The Multinomial Naive Bayes classifier applies Bayes' theorem to learn: *"Given this word appeared, how much does it shift the probability toward FAKE?"* It mines conditional probability patterns across all 44,000 documents — building a probability table for every term in the vocabulary.

Phase 5 — Evaluation. We split the data 80/20 — stratified by label — and measure accuracy, precision, recall, F1-score, and the confusion matrix. Stratification is critical: without it, a random split might over-represent one class and produce misleading metrics.

I'll now hand it over to [Member 3] to explain the NLP pipeline and similarity-based verification.

MEMBER 3 — NLP Pipeline, Cosine Similarity & Dual-Model Design

 (5:00 – 7:30)

Thank you [Member 2]. I'm [Member 3], and I'll walk you through how TruthLens actually processes text — from raw input to prediction.

The NLP Preprocessing Pipeline

Before any machine learning happens, raw text must be cleaned and structured. Our `TextPreprocessor` module handles this in **seven steps**, and the frontend visualizes every step:

Step 1 — Lowercasing. Ensures "Breaking" and "breaking" are treated as the same token.

Step 2 — Noise Removal. Strips URLs, HTML tags, emails, and special characters using regex. These add zero semantic value to classification.

Step 3 — Tokenization. Using NLTK's `word_tokenize`, the sentence is split into individual tokens.

Step 4 — Stop-word Removal. Common words like "the", "is", "at" are removed — but we deliberately **preserve negation words** like "not", "no", "never" — because "not guilty" and "guilty" carry opposite meanings.

Step 5 — Lemmatization. Words are reduced to their root form using WordNet. "running" → "run", "denied" → "deny". This reduces vocabulary size and improves generalization.

Step 6 — Mode-Aware Pipeline. For headlines, we skip stop-word removal entirely — headlines are already short, and removing words from a 6-word sentence destroys context. This required training two separate TF-IDF + Naive Bayes models with different configurations.

Step 7 — Rejoining. Tokens are rejoined into a clean string, ready for vectorization.

TF-IDF Vectorization & Classification

The clean text is vectorized into a sparse feature matrix and passed to the **Multinomial Naive Bayes classifier**, which outputs P(REAL) and P(FAKE). Our **threshold decision engine** then applies:

- $P(\text{FAKE}) \geq 80\% \rightarrow \text{FAKE}$
- $P(\text{REAL}) \geq 80\% \rightarrow \text{REAL}$
- Otherwise $\rightarrow \text{SUSPICIOUS}$ — indicating model uncertainty

The SUSPICIOUS class is a deliberate design choice — it prevents overconfident wrong predictions.

Cosine Similarity — Database Verification

Beyond the classifier, TruthLens runs a second check: **TF-IDF cosine similarity** against our live scraped headline database. This is a classic information retrieval technique from data mining.

The user's query and all 443 scraped headlines are vectorized together into a single TF-IDF matrix. We then compute the cosine similarity between the query vector and every headline vector. The cosine of the angle between two vectors measures how similar their term distributions are — 1.0 means identical, 0.0 means completely unrelated.

We apply three thresholds:

- $\geq 60\%$ similarity $\rightarrow \text{VERIFIED}$ — the claim matches a known real headline
- 35–59% $\rightarrow \text{PARTIAL MATCH}$ — a related topic was found
- $< 35\%$ $\rightarrow \text{UNVERIFIED}$ — no supporting evidence in the database

This dual-layer design — a classifier for pattern recognition, and cosine similarity for factual cross-referencing — is what makes TruthLens more robust than a single-model approach.

I'll now hand over to [Member 4] for the demo and results.

MEMBER 4 — System Demo, Multilingual Support, Results & Conclusion

 (7:30 – 10:00)

Thank you [Member 3]. I'm [Member 4], and I'll show you TruthLens in action.

System Architecture & Features

TruthLens is a full-stack application — a **React + TypeScript frontend** built with Vite, and a **Python Flask REST API backend** on port 5000. The two communicate via JSON over HTTP.

The backend exposes:

- **POST /api/predict** — runs the full NLP + classification + DB verification pipeline
- **GET /api/metrics** — returns model evaluation stats
- **GET /api/health** — reports model and DB status

New features in this version:

- **Dual-Model Design** — separate Naive Bayes models for articles and headlines, each with optimized TF-IDF parameters
- **Multi-Source News Scraper** — `bulk_fetch.py` populates the DB from 8 live RSS sources + Inshorts HTML scraping, achieving 400+ headlines
- **Kannada Language Support** — input is auto-detected using `langdetect`; Kannada text is translated to English using `deep-translator` with chunked processing for long articles before classification
- **Live SQLite Database** — scraped headlines are stored, deduplicated, and indexed for fast cosine similarity lookups

The UI has four tabs:

- **Detect** — paste any headline or article, get instant verdict with preprocessing visualization
- **Metrics** — accuracy, precision, recall, F1-score, confusion matrix
- **How It Works** — step-by-step NLP explainer
- **History** — session-level prediction log

(Demo: Paste a fake news headline → show FAKE verdict with confidence and preprocessing steps) (Demo: Paste a real news headline → show REAL verdict) (Demo: Paste Kannada text → show translation badge + verdict)

Model Performance

Here are our evaluation metrics on the held-out 20% test set:

Metric	Article Model	Headline Model
Accuracy	~92%	~89%
Precision	~91%	~88%
Recall	~90%	~87%
F1-Score	~90.5%	~87.5%

These numbers are strong for a classical ML approach — no deep learning, no GPU. The gains come entirely from rigorous data preprocessing and TF-IDF feature engineering.

The confusion matrix shows **false positives** — labeling real news as fake — are kept low. This matters: suppressing genuine information is as dangerous as spreading fake news.

Advantages & Limitations

Advantages:

- End-to-end data pipeline — from raw CSV and live web scraping to trained classifier

- Dual-layer verification: Naive Bayes classification + cosine similarity against live DB
- Fast inference — under 50ms per prediction
- Fully interpretable — every prediction traceable to word probabilities
- Multilingual support via automatic language detection and translation
- No GPU or cloud dependency

Limitations:

- Sensitive to out-of-vocabulary terms — novel political jargon may not be captured
- Scraper coverage depends on feed availability — some sources may go offline
- The Kaggle dataset is primarily English-language American political news

Conclusion

TruthLens demonstrates that **data mining, web scraping, and classical NLP** are not just academic concepts — they are powerful, practical tools for solving real-world misinformation problems.

We built a complete data pipeline: from bulk data collection via 8 scraping sources, through a 7-step NLP preprocessing pipeline, TF-IDF feature extraction with bigrams, dual Naive Bayes classification, and cosine similarity-based database cross-referencing — all exposed through an interactive, multilingual web interface.

With over 90% accuracy on 44,000 labeled articles, and a live headline database of 443+ scraped entries, TruthLens is a working proof-of-concept for AI-powered fact-checking.

Thank you. We are happy to take any questions.

Timing Summary

Member	Section	Duration
Member 1	Introduction & Problem Statement	1 min 30 sec
Member 2	Dataset, Scraping & Data Mining Core	3 min 30 sec
Member 3	NLP Pipeline, Cosine Similarity & Dual-Model	2 min 30 sec
Member 4	Demo, Results & Conclusion	2 min 30 sec
Total		~10 min

Key Data Mining Concepts — Quick Reference

Concept	Where Used in TruthLens
Data Collection	Kaggle CSV + 8-source RSS/HTML scraper
Web Scraping	<code>news_fetcher.py</code> (BeautifulSoup), <code>bulk_fetch.py</code> (XML + BS4)
Data Cleaning	<code>TextPreprocessor</code> — regex, tokenization, stopwords, lemmatization
Feature Engineering	TF-IDF with unigrams + bigrams, <code>max_features=5000</code>

Concept	Where Used in TruthLens
Classification	Multinomial Naive Bayes — probabilistic text classification
Stratified Sampling	80/20 train-test split with <code>stratify=labels</code>
Model Evaluation	Accuracy, Precision, Recall, F1, Confusion Matrix
Similarity Search	TF-IDF Cosine Similarity for live DB cross-verification
Deduplication	SQLite INSERT with title uniqueness check
Multilingual NLP	Language detection + chunked Kannada → English translation